

Задание 3.2. Пошаговый алгоритм создания пользователя на бэкенде

Исходные данные

- Бэкенд получает HTTP-запрос `POST /api/v1/register` с JSON-телом, содержащим поля: `firstName`, `lastName`, `userName`, `password`, `recaptchaToken`.
- Задача: проверить все данные, при успехе создать запись в БД и вернуть ответ `201 Created`.

Пошаговый алгоритм

Шаг 1. Приём и парсинг запроса

- Извлечь тело запроса (строку JSON).
- Если тело отсутствует или JSON невалидный → ответ `400 Bad Request` с сообщением `"Invalid request body"`.
- Преобразовать JSON в объект/структуру данных.

Шаг 2. Проверка наличия всех обязательных полей

- Обязательные поля: `firstName`, `lastName`, `userName`, `password`, `recaptchaToken`.
- Для каждого поля проверить, что оно присутствует и не является пустой строкой (после `trim()`).
- Если какое-то поле отсутствует или пустое → ответ `400` с сообщением `"Missing required field: <fieldName>"`.

Шаг 3. Валидация формата и ограничений полей

- **firstName**
 - Разрешены только буквы (латиница или кириллица). Регулярное выражение: `^[A-Za-zА-Яа-я]+$`.
 - Длина от 1 до 50 символов.
 - При несоответствии → `400` с сообщением `"First name must contain only letters (1–50 characters)"`.
- **lastName** – аналогично `firstName`.
- **userName**
 - Разрешены латиница, цифры, символ подчёркивания. Регулярка: `^[A-Za-z0-9_]+$`.
 - Длина от 3 до 30 символов.
 - При несоответствии → `400` с сообщением `"Username must be 3–30 characters and can contain letters, digits, underscore"`.

- **password**
 - Проверить длину: минимум 8 символов.
 - Проверить наличие хотя бы одной цифры (0-9).
 - Проверить наличие хотя бы одной заглавной буквы (A-Z).
 - Проверить наличие хотя бы одной строчной буквы (a-z).
 - Проверить наличие хотя бы одного специального символа (не буква и не цифра, например !@#\$%^&*).
 - Если любое условие не выполнено → ответ 400 с сообщением (точно как в интерфейсе):
"Passwords must have at least one non alphanumeric character, one digit ('0'-'9'), one uppercase ('A'-'Z'), one lowercase ('a'-'z'), one special character and Password must be eight characters or longer."
- **recaptchaToken**
 - Проверить, что токен не пустая строка.
 - Если токен отсутствует или пустой → 400 с сообщением "Please verify reCaptcha to register!".

Шаг 4. Валидация reCAPTCHA токена через сервер Google

- Бэкенд отправляет синхронный HTTP-запрос (POST) к API Google:
<https://www.google.com/recaptcha/api/siteverify>
Параметры:
 - **secret** – секретный ключ reCAPTCHA (хранится на бэкенде).
 - **response** – полученный recaptchaToken.
 - (опционально) **remoteip** – IP адрес пользователя.
- Получить ответ от Google в формате JSON.
- Проверить поле "success" – должно быть true.
- Если success равен false → ответ 400 с сообщением "Please verify reCaptcha to register!".
- (Дополнительно можно проверить поле "hostname" – должно совпадать с вашим доменом для защиты от подделки токена.)

Шаг 5. Проверка уникальности userName в базе данных

- Выполнить запрос к таблице пользователей:
SELECT id FROM users WHERE userName = ? (передать значение userName).
- Если запись найдена (т.е. userName уже занят) → ответ 409 Conflict с сообщением "User exists!".
- Проверку лучше делать после reCAPTCHA, чтобы не тратить ресурсы Google на заведомо дублирующиеся имена, но допустим и до – на усмотрение. Однако, если reCAPTCHA не пройдена, проверка уникальности не имеет смысла, так как мы всё равно вернём ошибку.

Шаг 6. Хэширование пароля

- Использовать криптостойкий алгоритм хэширования с солью (например, bcrypt, Argon2 или PBKDF2).
- Сгенерировать соль (bcrypt включает соль автоматически).
- Вычислить хэш: `hash = bcrypt.hash(password, saltRounds)` (обычно 10–12 раундов).
- Никогда не хранить пароль в открытом виде.
- Если хэширование завершилось ошибкой → ответ `500 Internal Server Error` с сообщением `"Internal server error"`.

Шаг 7. Создание записи пользователя в базе данных

- Подготовить SQL-запрос `INSERT INTO users (firstName, lastName, userName, passwordHash, createdAt, updatedAt) VALUES (?, ?, ?, ?, NOW(), NOW())`.
- Выполнить запрос в транзакции (для целостности данных).
- Получить сгенерированный `userId` (например, автоинкрементное поле).
- Если возникает ошибка БД (нарушение уникальности – хотя мы уже проверили, может быть гонка; таймаут; другие ошибки) → откатить транзакцию и вернуть `500 Internal Server Error` с сообщением `"Failed to create user"`.

Шаг 8. Формирование успешного ответа

- Установить HTTP статус `201 Created`.
- Вернуть JSON:

```
{
  "userId": <полученный ID>,
  "userName": "<значение userName из запроса>",
  "message": "User Register Successfully"
}
```

Шаг 9. Логирование

- Записать в лог (например, INFO) факт успешной регистрации: `"User registered: userName=ivan, userId=42"`.
- Никогда не логировать пароль или его хэш.